

OpsIntel Copilot

AWS + Databricks + AWS-hosted RAG Investigation Platform

This is the strongest version for industry-readiness:

Everything cloud-based on AWS

+ Databricks for lakehouse processing

+ RAG assistant hosted on AWS

+ Streamlit/FastAPI dashboard hosted on AWS

No local-first architecture, except your laptop is only for writing code and pushing to GitHub.

Final architecture choice

Cloud stack

Layer	Service
Raw storage	AWS S3
Lakehouse processing	Databricks on AWS
Table format	Delta Lake
Orchestration	Databricks Workflows / Jobs
Metadata database	Amazon RDS PostgreSQL
Vector/RAG layer	Amazon Bedrock Knowledge Bases or Amazon OpenSearch Serverless / pgvector
LLM	Amazon Bedrock
Backend API	FastAPI on AWS ECS Fargate or App Runner
Dashboard UI	Streamlit on AWS ECS Fargate / App Runner / EC2
Container registry	Amazon ECR
Logs/monitoring	Amazon CloudWatch
Infrastructure as Code	Terraform
Code repo	GitHub

Databricks on AWS supports production workflow orchestration through Lakeflow Jobs, which are designed to coordinate and schedule repeatable data-processing workloads. Amazon Bedrock Knowledge Bases are AWS's managed RAG option: they retrieve relevant information from your data sources and augment the model prompt so generated responses are grounded in that data.

What runs where?

1. Your laptop

Only for development:

VS Code

Git

Terraform commands

Docker build commands

GitHub commits

Your laptop is **not** running the actual project.

2. AWS S3

Stores all operational data.

s3://opsintel-copilot/raw/

s3://opsintel-copilot/bronze/

s3://opsintel-copilot/silver/

s3://opsintel-copilot/gold/

s3://opsintel-copilot/rag-docs/

s3://opsintel-copilot/checkpoints/

Used for:

raw order data

raw security logs

pipeline logs

cleaned Delta tables

incident summaries

RAG documents

playbooks/runbooks

Delta Lake is a strong fit here because it supports lakehouse features like ACID transactions, schema enforcement, scalable metadata handling, and time travel on object storage such as S3.

3. Databricks on AWS

Databricks is your main processing engine.
It handles:

PySpark transformations

Delta Lake bronze/silver/gold tables

data reliability checks

security log processing

incident timeline generation

correlation engine jobs

Databricks reads from S3 raw data and writes curated Delta tables back to S3.

Flow:

S3 raw



Databricks bronze jobs



Databricks silver jobs



Databricks gold jobs



RDS + S3 curated outputs

Use **Databricks Workflows / Jobs** to schedule the pipeline instead of local Airflow for the final cloud version.

4. Amazon RDS PostgreSQL

RDS stores operational metadata.

Tables:

pipeline_runs
data_quality_results
security_alerts
incidents
incident_events
correlations
correlation_evidence
recommendations
rag_query_history

Why RDS?

Because your dashboard and API need fast structured access to the latest incidents, timelines, alerts, and correlation results.

5. RAG layer on AWS

You have two strong options.

Recommended option: Amazon Bedrock Knowledge Bases

Use:

Amazon Bedrock

Bedrock Knowledge Bases

S3 as data source

**OpenSearch Serverless or managed vector store behind
Knowledge Base**

Flow:

S3 rag-docs/



Bedrock Knowledge Base sync



User question



Retrieve relevant chunks



Bedrock model generates answer

This is the most AWS-native and industry-ready RAG choice. AWS documentation describes Knowledge Bases as implementing the RAG workflow from ingestion to retrieval and prompt augmentation. Use Bedrock Knowledge Bases for:

pipeline runbooks
security playbooks
incident summaries
correlation records
schema drift explanations
MITRE-style notes
data quality documentation

Alternative option: Custom RAG on AWS

Use:

FastAPI
Amazon Bedrock embeddings
Amazon OpenSearch Serverless vector search
or RDS PostgreSQL with pgvector

This gives you more engineering control, but takes longer.

My recommendation:

Use **Bedrock Knowledge Bases first**, then optionally build a custom retriever later if you want extra control.

6. FastAPI backend on AWS

Host FastAPI using one of these:

AWS App Runner
or
ECS Fargate

FastAPI handles:

dashboard APIs
incident queries
timeline queries
correlation queries
RAG question endpoint
Databricks job trigger endpoint

Example API endpoints:

GET /incidents
GET /incidents/{id}/timeline
GET /correlations
GET /data-quality/latest
GET /security-alerts/latest
POST /copilot/ask
POST /databricks/run-job

Use **ECS Fargate** if you want the most industry-style container deployment.
Use **App Runner** if you want simpler AWS deployment.

7. Streamlit dashboard on AWS

Host Streamlit on:

ECS Fargate
or
App Runner
or
EC2

Recommended:

ECS Fargate

Why?

Because it gives you strong AWS/container experience:

Docker

ECR

ECS service

Fargate task

Application Load Balancer

CloudWatch logs

AWS has sample patterns for deploying Streamlit on ECS/Fargate with an ALB and CloudFront/Cognito-style setup, so this is a realistic AWS deployment path.

Dashboard pages:

1. Overview

2. Data Reliability

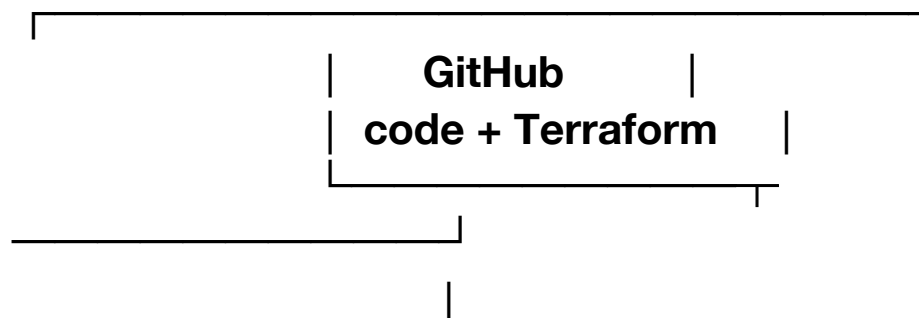
3. Security Intelligence

4. Incident Timeline

5. Correlation Engine

6. Investigation Copilot

Final cloud architecture





**Terraform / IaC
creates AWS infra**



AWS

S3 raw/



Databricks on AWS

logs/docs

PySpark + Delta

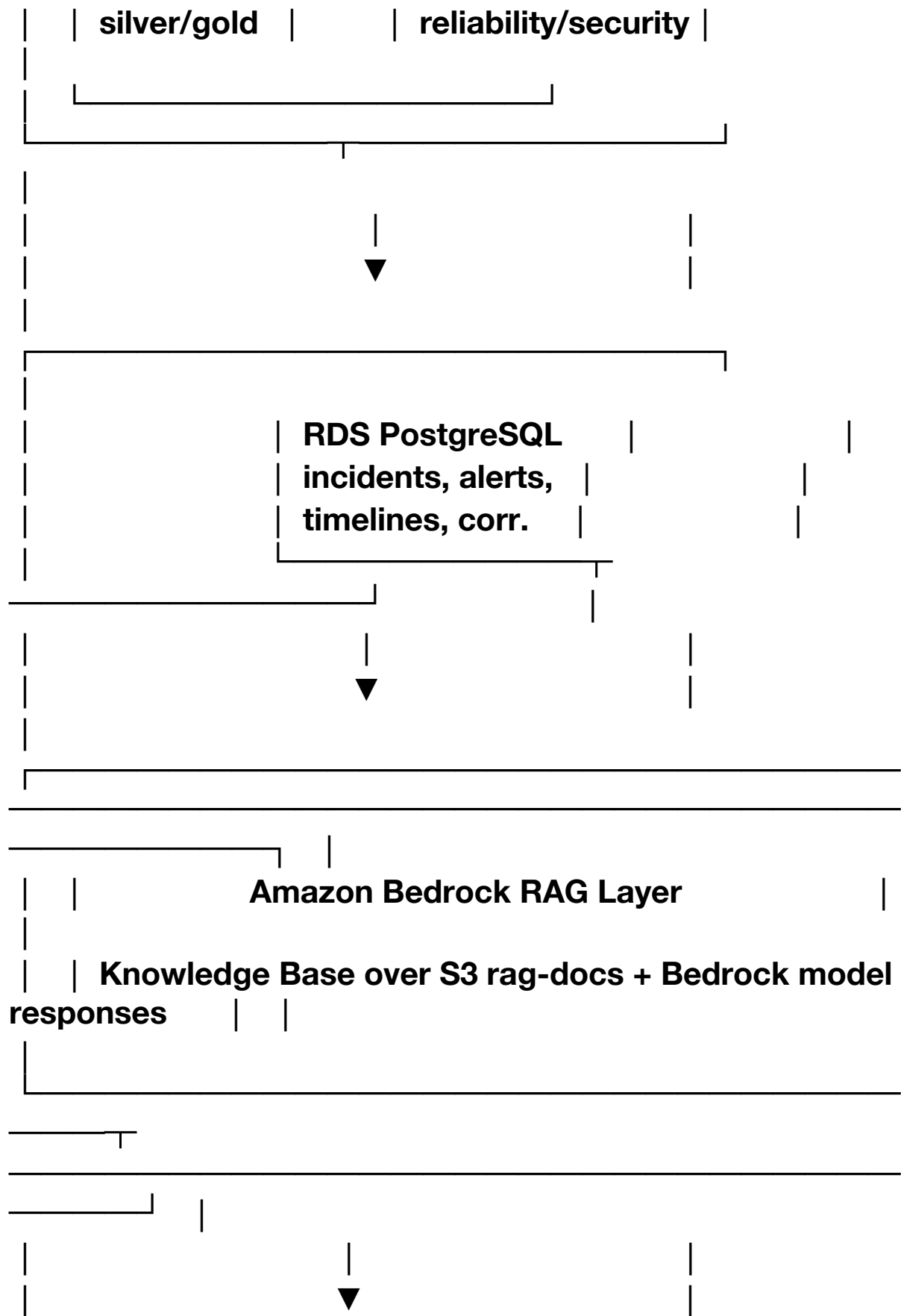
Workflows/Jobs

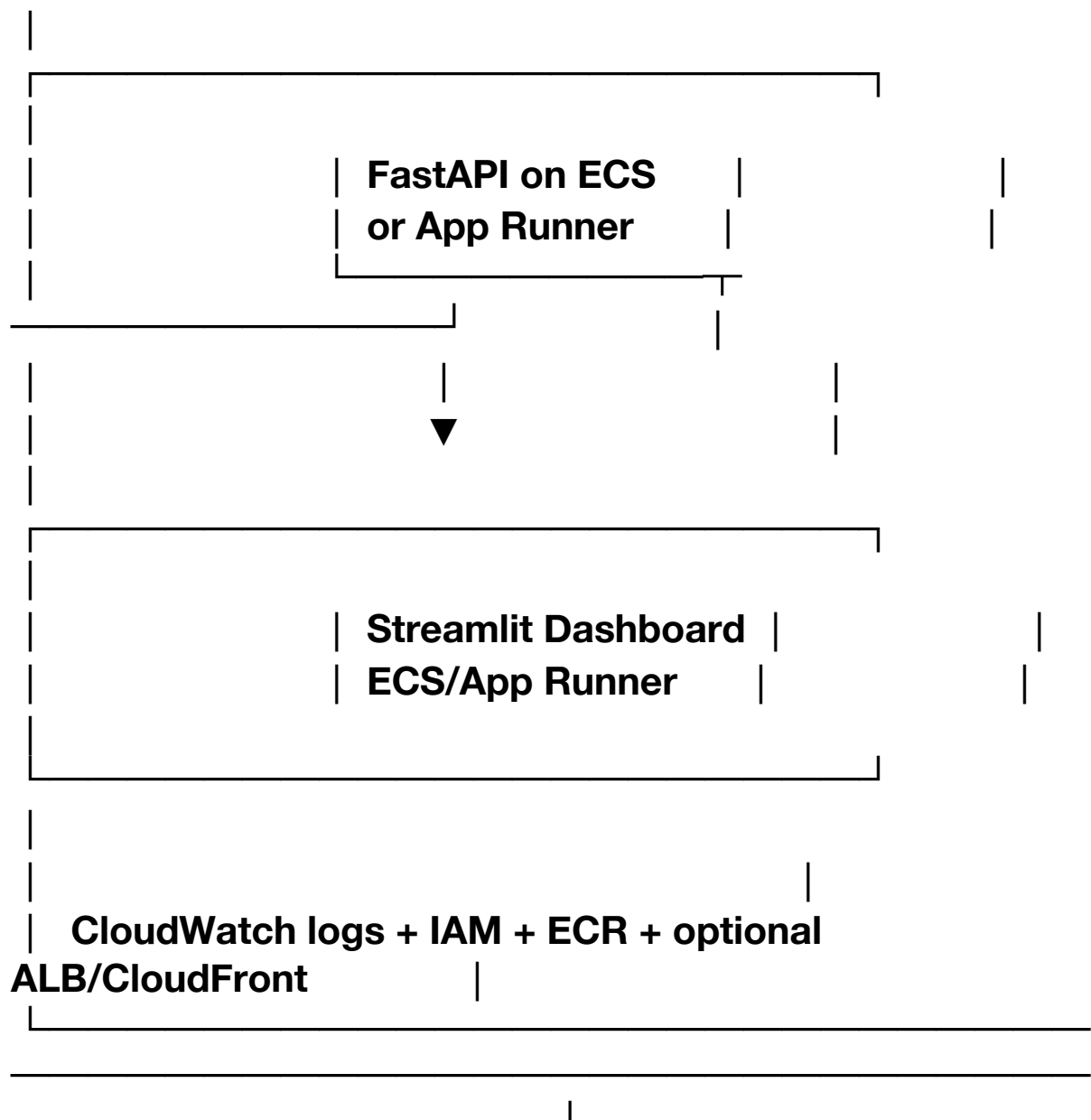


S3 bronze/



Delta Lake Tables





How the full system works

Step 1: Data arrives in S3

Your generators create:

orders data

pipeline logs

security logs

admin activity logs

API token events
warehouse export events

These are uploaded to:

S3 raw/

You can generate them using:

AWS Lambda
or
ECS scheduled task
or
Databricks notebook job

For maximum AWS usage, use:

ECS scheduled task or Lambda

Step 2: Databricks processes raw data

Databricks jobs read raw files from S3.
It creates:

bronze Delta tables = structured raw data
silver Delta tables = cleaned validated data
gold Delta tables = incidents, alerts, quality summaries

Databricks runs:

data reliability checks
security detection rules
timeline builder
correlation engine

File-arrival triggers can also run Databricks jobs when new files arrive in S3-like external locations.

Step 3: Data reliability module

Checks:

schema drift
row count anomaly
null spike
duplicates
freshness failure
bad record ratio
invalid currency
negative amount

Outputs:

data_quality_results
pipeline_runs
bad_records_summary
incident_events

Stored in:

Delta gold tables
RDS PostgreSQL
S3 rag-docs as JSON/text summaries

Step 4: Security intelligence module

Checks:

brute-force login

impossible travel
unusual admin login
API token rotation
privilege escalation
large data export
service account anomaly
config change near failure

Outputs:

security_alerts
security_events
incident_events

Stored in:

Delta gold tables
RDS PostgreSQL
S3 rag-docs

Step 5: Incident timeline engine

Databricks or FastAPI builds timeline records:

02:03 - unusual admin login
02:05 - API token rotated
02:07 - ingestion config changed
02:10 - pipeline started
02:14 - pipeline failed
02:15 - row count anomaly detected
02:18 - correlation created

Stored in:

RDS incident_events
S3 rag-docs/incidents/

Step 6: Correlation engine

Correlation rules connect events.

Example:

IF pipeline failure happens within 30 minutes of suspicious admin config change
THEN create security_to_pipeline correlation

Stores:

correlation_id
incident_id
linked_data_event
linked_security_event
confidence_score
evidence_ids
recommendation

This is one of your strongest technical features.

Step 7: RAG sync

Incident summaries, timeline documents, quality results, security alerts, and playbooks go to:

S3 rag-docs/

Bedrock Knowledge Base syncs those documents.

Then the user asks:

“Why did the orders pipeline fail and was it security related?”

Bedrock retrieves relevant incident/timeline/correlation/playbook chunks and generates an evidence-backed answer.

Step 8: Dashboard on AWS

Streamlit calls FastAPI.

FastAPI reads from:

RDS PostgreSQL

Bedrock Knowledge Base

Databricks job status APIs

S3 summaries

User sees:

system overview

data reliability failures

security alerts

incident timeline

correlation engine output

RAG copilot answer

Best service choices

For first full cloud version

Use this:

S3

Databricks on AWS

RDS PostgreSQL

Bedrock Knowledge Bases

Bedrock model

ECR

ECS Fargate

CloudWatch

Terraform

GitHub

This is the most industry-ready version.

Avoid initially

Do not add these at first unless time remains:

Kinesis

MSK Kafka

OpenSearch custom vector store

CloudFront

Cognito

Step Functions

Lambda-heavy orchestration

Redshift

They are useful, but they can make the project too wide.

Revised repo structure for full AWS version

opsintel-copilot/

```
|
|----- README.md
|----- .gitignore
|----- .env.example
|
|----- infra/
|   |----- terraform/
|   |   |----- main.tf
|   |   |----- variables.tf
|   |   |----- outputs.tf
|   |   |----- s3.tf
|   |   |----- rds.tf
|   |   |----- ecr.tf
```

```

|   |   |── ecs_fastapi.tf
|   |   |── ecs_streamlit.tf
|   |   |── iam.tf
|   |   |── bedrock.tf
|   |   |── cloudwatch.tf
|   |   |
|   |   |── diagrams/
|   |   |   |── aws_architecture.png
|   |   |
|   |   |── databricks/
|   |   |   |── notebooks/
|   |   |   |   |── 01_bronze_ingestion.py
|   |   |   |   |── 02_silver_data_quality.py
|   |   |   |   |── 03_security_detection.py
|   |   |   |   |── 04_incident_timeline.py
|   |   |   |   |── 05_correlation_engine.py
|   |   |   |   |── 06_gold_outputs.py
|   |   |   |
|   |   |   |── jobs/
|   |   |   |   |── opsintel_job_config.json
|   |   |   |
|   |   |   |── sql/
|   |   |   |   |── create_delta_tables.sql
|   |   |   |   |── gold_quality_views.sql
|   |   |
|   |   |── services/
|   |   |   |── generators/
|   |   |   |   |── Dockerfile
|   |   |   |   |── generate_orders.py
|   |   |   |   |── generate_security_logs.py
|   |   |   |   |── generate_admin_events.py

```

```

├── scenario_runner.py
├── upload_to_s3.py
├── api/
│   ├── Dockerfile
│   ├── main.py
│   ├── routes_incidents.py
│   ├── routes_timeline.py
│   ├── routes_correlation.py
│   ├── routes_copilot.py
│   ├── bedrock_client.py
│   ├── rds_client.py
│   └── databricks_client.py
├── dashboard/
│   ├── Dockerfile
│   ├── app.py
│   ├── pages/
│   │   ├── 1_Overview.py
│   │   ├── 2_Data_Reliability.py
│   │   ├── 3_Security_Intelligence.py
│   │   ├── 4_Incident_Timeline.py
│   │   ├── 5_Correlation_Engine.py
│   │   └── 6_Investigation_Copilot.py
│   └── components/
│       ├── metric_cards.py
│       ├── timeline_view.py
│       ├── incident_table.py
│       └── copilot_chat.py
└── sql/

```

```
|   |   |— rds_schema.sql
|   |   |— seed_runbooks.sql
|   |   |— seed_security_playbooks.sql
|
|— docs/
|   |— architecture.md
|   |— aws_setup.md
|   |— databricks_setup.md
|   |— bedrock_rag_setup.md
|   |— demo_script.md
|   |— incident_scenarios.md
|   |— resume_bullets.md
|
|— sample_data/
|   |— orders/
|   |— security_logs/
|   |— admin_events/
|   |— playbooks/
|
|— tests/
|   |— test_data_quality.py
|   |— test_security_rules.py
|   |— test_correlation_logic.py
|   |— test_api_routes.py
```

What you can say on resume

Built **OpsIntel Copilot**, an AWS-native AI investigation platform using Databricks, S3, Delta Lake, RDS PostgreSQL, Bedrock Knowledge Bases, FastAPI, Streamlit, and ECS Fargate to diagnose data pipeline failures and security incidents with RAG-grounded evidence.

Implemented Databricks Lakehouse pipelines over S3 raw, bronze, silver, and gold layers to process operational data, security logs, data quality results, incident timelines, and correlation records.

Designed a cross-domain correlation engine to link data pipeline failures with suspicious security events, generating confidence-scored incident summaries and remediation recommendations.

This is a serious project. This is absolutely stronger than a local-only demo.

Final decision

Your stack is now:

AWS cloud-native

- + Databricks lakehouse**
- + Bedrock RAG**
- + ECS-hosted FastAPI**
- + ECS-hosted Streamlit**
- + RDS metadata store**
- + S3 raw/Delta/RAG storage**
- + Terraform infrastructure**

Final project title:

OpsIntel Copilot — AWS-Native AI Data Reliability & Security Investigation Platform

This gives you strong relevance for:

Amazon / AWS

Databricks roles

cloud data engineering

AI data engineering

security data engineering

consulting roles

data platform roles

GenAI/RAG engineering

Week 1 — Project foundation + AWS setup

Goal: create the repo, structure, basic AWS resources, and documentation foundation.

Build:

GitHub repo

Clean folder structure

.env.example

README draft

Architecture diagram draft

Terraform base setup

AWS S3 bucket structure

IAM roles/policies

Deliverables:

opsintel-copilot/

infra/terraform/

services/

databricks/

docs/

sample_data/

By end of week, you should have:

A clean GitHub repo + S3 raw/bronze/silver/gold/rag-docs structure.

Week 2 — Synthetic data generators

Goal: generate realistic operational data.

Build generators for:

orders data
pipeline logs
security logs
admin activity logs
API token events
warehouse export events
playbooks/runbooks

Important files:

services/generators/generate_orders.py
services/generators/generate_security_logs.py
services/generators/generate_admin_events.py
services/generators/scenario_runner.py
services/generators/upload_to_s3.py

Deliverables:

Raw JSON/CSV files uploaded to S3 raw/
Scenario-based failure data
Normal + suspicious activity data

By end of week, you should be able to say:
I created synthetic cloud operations data and uploaded it into AWS S3 as the raw ingestion layer.

Week 3 — Databricks bronze/silver pipeline

Goal: build the lakehouse ingestion and cleaning layer.
Build Databricks notebooks:

01_bronze_ingestion.py
02_silver_data_quality.py
create_delta_tables.sql

Implement:

- Read raw files from S3**
- Create bronze Delta tables**
- Clean data into silver Delta tables**
- Handle bad records**
- Enforce schema**
- Remove duplicates**
- Validate timestamps**

Deliverables:

- bronze_orders**
- bronze_security_logs**
- bronze_admin_events**

- silver_orders**
- silver_security_logs**
- silver_admin_events**
- bad_records_summary**

By end of week, you should be able to say:
I built Databricks Delta Lake bronze and silver pipelines over S3 using PySpark and data quality checks.

Week 4 — Data reliability module

Goal: detect pipeline/data issues.
Build checks for:

- schema drift**
- row count anomaly**
- null spike**
- duplicate records**

freshness failure
bad record ratio
invalid currency
negative amount

Create outputs:

data_quality_results
pipeline_runs
incident_events
bad_records_summary

Store output in:

Delta gold tables
RDS PostgreSQL
S3 rag-docs summaries

Deliverables:

03 or 04 reliability notebook
RDS schema tables
Gold quality views
Quality incident summaries

By end of week, this becomes your first strong “data engineering reliability” feature.

Week 5 — Security intelligence module

Goal: detect suspicious operational/security behavior.
Build rules for:

brute-force login
impossible travel

unusual admin login
API token rotation
privilege escalation
large data export
service account anomaly
config change near failure

Create outputs:

security_alerts
security_events
incident_events

Store in:

Delta gold tables
RDS PostgreSQL
S3 rag-docs

Deliverables:

03_security_detection.py
security_alerts table
security_events table
security playbook documents

By end of week, you'll have the "security data engineering" angle.

Week 6 — Incident timeline + correlation engine

Goal: this is the project's "wow" feature.

Build:

04_incident_timeline.py
05_correlation_engine.py

Create incident timeline like:

02:03 - unusual admin login

02:05 - API token rotated

02:07 - ingestion config changed

02:10 - pipeline started

02:14 - pipeline failed

02:15 - row count anomaly detected

02:18 - correlation created

Correlation logic:

IF suspicious admin/config event happens within 30 minutes of pipeline failure

THEN create security_to_pipeline correlation

Outputs:

incidents

incident_events

correlations

correlation_evidence

recommendations

By end of week, you should be able to say:

I designed a correlation engine that links data pipeline failures with suspicious security events and generates confidence-scored recommendations.

This is probably the strongest technical part of the project.

Week 7 — Bedrock RAG + FastAPI backend

Goal: make the investigation copilot work.

Build RAG flow:

S3 rag-docs/

↓

Bedrock Knowledge Base sync

↓

User question

↓

Retrieve relevant incident/playbook/timeline chunks

↓

Bedrock model generates grounded answer

Build FastAPI endpoints:

GET /incidents

GET /incidents/{id}/timeline

GET /correlations

GET /data-quality/latest

GET /security-alerts/latest

POST /copilot/ask

POST /databricks/run-job

Important files:

services/api/main.py

services/api/routes_incidents.py

services/api/routes_timeline.py

services/api/routes_correlation.py

services/api/routes_copilot.py

services/api/bedrock_client.py

services/api/rds_client.py

services/api/databricks_client.py

By end of week, you should be able to ask:

“Why did the orders pipeline fail, and was it security related?”

And the system should answer using incident evidence.

Week 8 — Streamlit dashboard + AWS deployment polish

Goal: make it demo-ready for recruiters.

Build dashboard pages:

- 1. Overview**
- 2. Data Reliability**
- 3. Security Intelligence**
- 4. Incident Timeline**
- 5. Correlation Engine**
- 6. Investigation Copilot**

Deploy:

FastAPI → ECS Fargate or App Runner

Streamlit → ECS Fargate or App Runner

Images → ECR

Logs → CloudWatch

Infra → Terraform

Finish docs:

architecture.md

aws_setup.md

databricks_setup.md

bedrock_rag_setup.md

demo_script.md

incident_scenarios.md

resume_bullets.md

By end of week, you should have:

A cloud-hosted AWS + Databricks + RAG project with dashboard, API, lakehouse pipeline, security/reliability intelligence, and GitHub documentation.

Best weekly priority order

Week 1: Repo + AWS foundation
Week 2: Data generators + S3 ingestion
Week 3: Databricks bronze/silver Delta pipeline
Week 4: Data reliability checks
Week 5: Security detection rules
Week 6: Timeline + correlation engine
Week 7: Bedrock RAG + FastAPI
Week 8: Streamlit + ECS deployment + README polish

Resume-ready milestone after each phase

Week 3: AWS + Databricks lakehouse project
Week 4: Data reliability engineering project
Week 5: Security analytics project
Week 6: Correlation engine / incident intelligence project
Week 7: RAG / GenAI data platform project
Week 8: Fully cloud-deployed portfolio project

My recommendation: build the **MVP in 4 weeks**, then polish/deploy in the next 4.

MVP must include:

S3

Databricks bronze/silver/gold

Data quality checks

Security alerts

Correlation engine

Basic FastAPI

Basic Streamlit

Full version adds:

Bedrock Knowledge Base

ECS Fargate deployment

Terraform

CloudWatch

Strong documentation

Demo video/screenshots

This is a very serious project. Built properly, it can become your strongest portfolio piece for **Data Engineering + AWS + Databricks + GenAI/RAG** roles.